

Content Addressing and Multi-Dimensional Shape Hashing

Review, Design, and Clean Phased Implementation Plan

Prepared for the Fe origin / trace instrumentation implementor agent
2026-05-24

Architectural thesis

Reintroduce the old multi-dimensional Merkle-style graph hashing, but as a derived per-phase content-address view over phase-owned identities. Do not let content addressing become another compiler identity system.

Contents

- 1. Executive summary
- 2. What survived, what was dropped, and what the old implementation actually did
- 3. Correct architectural target
- 4. Hash dimensions, elision, and content-address views
- 5. Canonical graph aggregation, DAGs, and cycles
- 6. Data model and API sketch
- 7. Implementation plan
- 8. Tests and acceptance criteria
- 9. Risks, caveats, trade-offs, and workarounds
- 10. Fuzzing, trace, CLI/LSP, Datalog, and proof-facing integrations
- Appendix A. Old implementation review notes
- Appendix B. Research notes and references
- Appendix C. Agent checklist

1. Executive summary

The current phased branch is right to be smaller and SSOT-compliant, but the content-addressing surface is currently too thin for the long-term goals. The old branch contained a real ShapeGraph model, multi-dimensional node digests, ordered tree aggregation, graph aggregation, and a strict ShapeDescribe derive macro. The new branch kept only small digest utilities. That was the right temporary cleanup move, but the generalized per-IR content-address view should come back in a cleaner shape.

Primary recommendation

Reintroduce generalized per-phase shape graphs in a new downstream shape-address layer. Phase crates build shape graphs from phase-owned origin keys; the shape layer computes derived hashes; trace/query/LSP/fuzzing consume hash facts. Common must not become a semantic registry.

The design should support four related but distinct use cases:

- **Granular fuzzing and deduplication:** bucket, mutate, and compare programs or IR fragments by structure, names, constants, types, and trace-event dimensions.
- **Compiler regression diagnosis:** tell whether a change is structural, naming-only, literal-only, type-only, or trace/instrumentation-only.
- **Agent and LSP probing:** let a tool ask “what changed here?” or “which loop shape is responsible?” using compact stable fingerprints.
- **Attestation and audit support:** produce content-addressable debug/trace artifacts that can be signed, stored, compared, and linked without claiming full semantic equivalence.

The most important correction to the old approach is cycle handling. A conventional Merkle DAG is acyclic by construction: parent identifiers depend on child identifiers, so cycles cannot be constructed directly. Cyclic IRs therefore need a cycle policy: either keep the Merkle child graph acyclic and model cycles as non-recursive graph edges, or condense strongly connected components before hashing, or use a labeled graph refinement approach for an anonymous structural hash. The default should be SCC condensation plus deterministic stable-key-aware component hashing; optional anonymous/WL-style structural hashes can be added for fuzzing and similarity search.

The new implementation should also replace the old 64-bit FNV digest with a real content-address digest. FNV is deterministic and fine for smoke tests, but not for durable content addressing, external artifacts, or security-adjacent attestations. Use domain-separated BLAKE3 internally or SHA-256/multihash for externally stable content IDs; make the digest algorithm and hash schema version explicit.

2. What survived, what was dropped, and what the old implementation actually did

This section is intentionally precise because there was some confusion during voice discussion about whether the full Merkle graph infrastructure survived. It did not. The current branch preserved the idea of dimensions and digest utilities, but it dropped the old graph object, per-node hash table, derive macro, and graph-wide aggregation.

Capability	Old origin-overhaul shape system	Current phased branch	Recommendation
Dimensions	Structure, Names, Constants, Types,	Same enum is present	Keep; add policy/version

	TraceEvents		metadata.
Node graph model	ShapeGraph with nodes, fields, ordered children, graph edges	No full ShapeGraph model	Reintroduce downstream of phase crates.
Local node digest	Per-dimension local digests from kind/fields	Utility function only	Keep but upgrade digest and canonical codec.
Tree/Merkle aggregation	Recursive child aggregation with ordered children	Utility function only	Restore per-node tree hashes.
Graph aggregation	Graph digest over sorted nodes and sorted graph edges	Only graph_edge_digest helper	Restore graph-wide and component-level hashes.
Cycles	Child cycles panic; graph-edge cycles are tolerated because graph edges are not recursively followed	No full cycle policy	Add explicit cycle policy; do not silently recurse.
Derive macro	ShapeDescribe proc macro requires per-field policy	Not present	Re-add later after core hash engine is stable.
Hash algorithm	Stable 64-bit FNV-1a	Stable 64-bit FNV-like digest in 16 hex chars	Use BLAKE3 or SHA-256 for durable content addressing.
SSOT posture	Useful, but common owned a broad semantic surface nearby	Cleaner branch; shape is too small	Reintroduce as derived view, never identity owner.

Old implementation findings

The old branch's public shape facade re-exported ShapeBuilder, ShapeDescribe, ShapeFieldValue, ShapeGraph, ShapeNode/Edge/Child, DimensionDigests, ShapeGraphHashes, ShapeNodeHashes, StableDigest, and the fe-shape-derive proc macro. That is evidence that the old design was more than a utility hash function: it was a graph description and aggregation layer.

The old ShapeGraph had stable keys, kinds, dimensioned fields, ordered child edges, and separate graph edges. It enforced non-empty stable keys, node kinds, field names, child labels, and edge labels, and rejected duplicate stable keys.

The old hash logic computed local per-dimension digests, then recursive ordered child/tree digests, then a graph digest over sorted nodes and sorted graph edges. Fields were sorted and therefore unordered metadata; children were ordered and label-sensitive in the structure dimension. Graph edges did not suppress child hashing, and tests asserted that full edge labels were observed.

Important correction

The old child/Merkle tree path did not support cycles. The old code explicitly asserted when the child graph contained a cycle. General graph edges could form cycles only because they were aggregated non-recursively after tree hashes had already been computed.

Current phased branch findings

The current branch retains ShapeDimension and the ideas of local, ordered-tree, and graph-edge digests, but it has no full graph object, no per-node hash table, no ShapeDescribe trait/derive, no graph-wide root hashes, and no cycle policy. This is acceptable as a cleanup baseline, but not enough for granular fuzzing or content-addressable IR views.

The current branch's digest type is a 16-character lowercase hex string. That implies a 64-bit digest. That is useful for deterministic tests and compact debugging, but it is too small for external content-addressed artifacts or cryptographic-adjacent audit workflows.

3. Correct architectural target

The shape/content-addressing layer should sit downstream of phase-owned compiler identity:

```
HIR / MIR / codegen / backend
  own phase-local nodes and OriginExportKey projections
    ↓
phase-owned shape adapters
  describe phase IR nodes using canonical shape fields/children/edges
    ↓
shape-address engine
  computes local/tree/component/graph hashes by dimension
    ↓
trace-facts / debug model / Datalog / CLI / LSP / fuzzing
  consume derived ShapeHash facts
```

The authority rule is simple:

SSOT rule

Shape hashes identify derived content views. They do not define compiler identity. Every shape node that corresponds to a compiler entity must be keyed by the phase-owned OriginExportKey for that entity.

Authority boundaries

Layer	Owns	Must not own
common::origin	Generic OriginKey/OriginExportKey mechanics	HIR/MIR/codegen semantics or shape policy for phase internals
phase crates	Phase-local identity, stable export projection, shape adapter for that phase	Global cross-phase content-address policy
shape-address	Generic shape graph data model, hash algorithms, cycle policies, validation	Compiler semantic meaning of HIR/MIR/codegen nodes
trace-facts	Serializable emitted/derived fact records	Canonical compiler state
trace-query/LSP/CLI	Reports, graph views, queries, fuzzing UX	Identity or semantics

Design goals

- Per-phase, not cross-phase: each IR level gets its own shape hash policy and graph view.
- Multi-dimensional: structure, names, constants, types, trace events, and optional future dimensions must remain independently queryable.
- Merkle-style where acyclic: every node should have a stable local hash and an aggregate hash over its reachable child content when the dependency relation is acyclic.
- Cycle-aware where necessary: cyclic CFG/dataflow graphs need an explicit SCC or graph-refinement policy, not recursion.
- Canonical and deterministic: hash results must not depend on map iteration order, allocation order, parallel scheduling, or platform serialization quirks.
- Versioned: algorithm, schema, dimension policy, and serialization version must be encoded into every durable hash.
- Derived: content addresses are views over compiler state, not replacements for phase-owned IDs.

Non-goals

- Do not claim semantic equivalence. A shape hash is a structural/content fingerprint under a declared policy.
- Do not build a global universal OriginNode enum in common.
- Do not reintroduce relation-table sprawl. Shape facts should be typed and relation rows generated downstream if needed.
- Do not make the first PR depend on a derive macro, Datalog, DWARF, or LSP integration.

4. Hash dimensions, elision, and content-address views

The multi-dimensional design is worth preserving because it gives us useful elision. We can compare the same IR while ignoring names, ignoring constants, focusing on type shape, or isolating trace-event/instrumentation changes. This is especially valuable for fuzzing and compiler regression triage.

Dimension	Should include	Should not include	Primary uses
Structure	Node kind, arity, child labels/order, graph edge labels/topology, block/control/data shape	User names, literal values, raw stable keys in anonymous mode	Find structural regressions; deduplicate alpha/literal variants; fuzz by IR topology
Names	Identifiers, symbols, field names when semantically name-bearing, function/contract names if requested	Numeric literals, type details unless represented by name only	Rename-aware diffs; alpha-renaming fuzz; UX display grouping
Constants	Literals, selectors, bytes, numeric constants, static gas constants if policy includes gas	Identifiers and type names	Literal fuzzing; constant-propagation regression detection
Types	Type constructor, bit	User variable names	Type-checking

	width, storage kind, ABI shape, pointer/location type where relevant	and literal values	regressions; type-directed fuzzing
TraceEvents	Compiler events, synthetic-origin classifications, storage decisions, optimization snapshots	User-authored syntax that belongs in structure/names/constants/types	Instrumentation regression detection; explain “why did this instruction exist?”

Elision examples

```

same_structure := graph_digest(Structure)
same_names     := graph_digest(Names)
same_literals  := graph_digest(Constants)
same_types     := graph_digest(Types)

```

```

rename-only change:
  Structure equal
  Constants equal
  Types equal
  Names different

```

```

literal-only change:
  Structure equal
  Names equal
  Types equal
  Constants different

```

```

control-flow change:
  Structure different
  other dimensions may or may not differ

```

One caveat in the old implementation is that graph aggregation wrote stable keys into the graph digest for every dimension. That makes the structure digest identity-sensitive even when the intended query is anonymous structural shape. The new design should split identity-bound and anonymous views rather than mixing both into one digest.

View	Includes stable keys?	Purpose
Identity-bound content address	Yes, via canonical OriginExportKey or shape node key	Durable artifact fingerprint, trace/debug linking, exact compiler snapshot comparison
Anonymous structural shape	No, except as deterministic tie-breaker outside the hashed payload if needed	Alpha/renaming-insensitive comparison, fuzzing buckets, generalized shape search
Display/debug hash	Maybe, but labeled as display-only	Short fingerprints for logs/LSP, not durable external CIDs

Canonical serialization

Do not use process-randomized HashMap order, Debug formatting, or ad hoc JSON serialization for durable content addresses. Use a small tagged length-delimited binary codec for internal hashing. If shape facts are exported as JSON and signed or re-hashed externally, use a real canonical JSON profile such as RFC 8785/JCS or avoid JSON and use a canonical CBOR-like format.

```

hash_input :=
  magic("fe.shape")
  schema_version(u32)
  algorithm("blake3-256" | "sha2-256")
  phase_level("hir" | "mir" | "sonatina" | "bytecode")
  dimension("structure" | "names" | ...)
  view_mode("identity_bound" | "anonymous")
  record_tag("node.local" | "node.tree" | "scc" | "graph")
  length_delimited_fields(...)

```

5. Canonical graph aggregation, DAGs, and cycles

Trees and DAGs are easy to Merkleize. General cyclic graphs are not. A Merkle DAG node hash depends on child hashes, so cycles have no bottom-up construction order. The design therefore needs separate policies for acyclic children, graph edges, and cyclic dependency edges.

Recommended edge taxonomy

Edge type	Meaning	Aggregation behavior
Child edge	Ordered containment or syntax/IR ownership relation; forms a tree/DAG	Merkle-recursive. Must be acyclic unless cycle policy explicitly handles SCCs.
Graph edge	CFG/dataflow/call/reference edge within an IR graph	Aggregated as sorted edge records using endpoint hashes; may be cyclic because it is not followed recursively.
Dependency edge	Optional selected relation that should contribute recursively even if not containment	Use SCC condensation if cycles are possible.
Origin edge	Cross-phase or causal provenance relation	Usually excluded from per-phase shape hash; include only in a separate trace/origin view.

Cycle policy options

Policy	How it works	Pros	Cons / caveats	Recommended use
Reject cycles	Detect recursion and error on cyclic child/dependency graph	Simple, honest, matches old child path	Not enough for CFG/dataflow graphs if modeled recursively	AST/HIR trees and early PRs
Keep cycles as graph edges	Merkleize acyclic ownership tree; aggregate cyclic CFG/dataflow edges after node tree hashes	Very stable and simple; likely enough for many IRs	Node aggregate does not include arbitrary cyclic reachability recursively	Default for CFG-like IRs initially
SCC condensation	Tarjan SCCs over selected dependency edges;	Handles cycles deterministically; clear component-	Per-node hashes inside a symmetric SCC need careful	Recommended first cycle-aware

	hash each SCC as a component; hash condensation DAG bottom-up	level addresses	role labeling	implementation
WL refinement	Iteratively hash labeled neighborhoods; can produce node/subgraph hashes in cyclic graphs	Good for anonymous structural bucketing and fuzzing	Not a proof of exact graph isomorphism; version/iteration sensitive	Optional fuzzing/search mode
Exact canonical labeling	Use a canonical graph labeling/isomorphism algorithm for exact anonymous graph hash	Most rigorous for graph equivalence under renaming	Complex, slower, harder to maintain	Later if WL is insufficient

Default cycle design: SCC condensation plus edge aggregation

The recommended default has two complementary layers:

1. Compute local per-node digests for every dimension from node kind and dimensioned fields.
2. Compute tree/DAG digests over acyclic child edges. If a child cycle exists and the policy is Reject, fail validation. If policy is CondenseScc, collapse SCCs first.
3. For each SCC, compute a component digest from sorted member local/tree digests and sorted internal edge records. Use full edge labels and both directions as encoded.
4. Build the condensation DAG and compute component aggregate digests bottom-up.
5. Compute graph-level digests from sorted components, sorted cross-component edges, and selected node/component aggregate hashes.
6. For per-node cyclic-context hashes, attach the containing SCC digest and an optional role digest; do not pretend a node has a unique recursive Merkle hash if it is structurally symmetric inside a cycle.

```

fn hash_shape_graph(graph, policy):
    validate_keys_and_labels(graph)
    local[node][dim] = hash_local(node, dim, policy)

    if policy.child_cycles == Reject:
        tree[node][dim] = merkle_tree_hash_acyclic_children(graph, local)
    else:
        sccs = tarjan(graph.selected_recursive_edges())
        component_local[scc][dim] = hash_scc_members_and_internal_edges(scc, local, policy)
        component_tree[scc][dim] = hash_condensation_dag(sccs, component_local, policy)
        tree[node][dim] = hash_node_with_component_context(node, component_tree[scc(node)]),
        policy)

    graph_digest[dim] = hash_sorted_nodes_components_and_edges(graph, tree, policy)
    return ShapeGraphHashes { node_hashes, component_hashes, graph_digest }

```

Stable-key-aware vs anonymous canonicalization

The compiler has stable origin/export keys. That makes deterministic hashing much easier. But fuzzing also wants a structural bucket that ignores identity. Support both explicitly.

Mode	Sorting/tie-breaking	Hash payload includes node keys?	Equivalent when
identity_bound	Sort by canonical ShapeNodeKey/OriginExportKey	Yes	Same compiler entities and same selected content under policy
anonymous_shape	Sort by canonical labels; refine ties with WL-style labels or exact canonicalization	No	Same structural shape under the chosen approximation/exact algorithm

6. Data model and API sketch

Use a small crate or module that is generic and downstream. Suggested name: crates/shape-address or crates/common/src/shape only if kept strictly generic. The former is safer.

```
pub enum ShapeDimension {
    Structure,
    Names,
    Constants,
    Types,
    TraceEvents,
}

pub enum ShapeViewMode {
    IdentityBound,
    AnonymousShape,
}

pub enum ShapeCyclePolicy {
    Reject,
    NonRecursiveGraphEdges,
    CondenseScg,
    WlRefine { iterations: u32 },
}

pub struct ShapeHashPolicy {
    pub schema_version: u32,
    pub algorithm: ShapeDigestAlgorithm,
    pub level: ShapeLevel, // "hir", "mir", "sonatina", ...
    pub dimensions: BTreeSet<ShapeDimension>,
    pub view_mode: ShapeViewMode,
    pub cycle_policy: ShapeCyclePolicy,
}

pub enum ShapeNodeKey {
    Entity(OriginExportKey),
    Derived { owner: OriginExportKey, local: ShapeLocalKey },
}

pub struct ShapeNode {
    pub key: ShapeNodeKey,
    pub kind: ShapeKind,
    pub fields: Vec<ShapeField>,
}

pub struct ShapeChild {
```

```

    pub parent: ShapeNodeKey,
    pub label: ShapeEdgeLabel,
    pub ordinal: u32,
    pub child: ShapeNodeKey,
}

pub struct ShapeEdge {
    pub source: ShapeNodeKey,
    pub label: ShapeEdgeLabel,
    pub target: ShapeNodeKey,
    pub role: ShapeEdgeRole,           // control, data, reference, call, ...
}

pub struct ShapeGraph {
    pub graph_key: ShapeGraphKey,
    pub nodes: BTreeMap<ShapeNodeKey, ShapeNode>,
    pub children: Vec<ShapeChild>,
    pub edges: Vec<ShapeEdge>,
}

pub struct ShapeNodeHashes {
    pub local: DimensionDigests,
    pub tree: DimensionDigests,
    pub component: Option<DimensionDigests>,
}

pub struct ShapeGraphHashes {
    pub policy_id: ShapePolicyId,
    pub nodes: BTreeMap<ShapeNodeKey, ShapeNodeHashes>,
    pub components: Vec<ShapeComponentHash>,
    pub graph: DimensionDigests,
}

```

Builder and adapter API

```

pub trait ShapeSink {
    fn add_node(&mut self, key: ShapeNodeKey, kind: ShapeKind) -> Result<()>;
    fn add_field(&mut self, node: &ShapeNodeKey, dim: ShapeDimension, name: FieldName, value:
ShapeValue) -> Result<()>;
    fn add_child(&mut self, parent: &ShapeNodeKey, label: ShapeEdgeLabel, ordinal: u32, child:
&ShapeNodeKey) -> Result<()>;
    fn add_edge(&mut self, source: &ShapeNodeKey, label: ShapeEdgeLabel, target: &ShapeNodeKey,
role: ShapeEdgeRole) -> Result<()>;
}

// Phase-owned adapter example.
pub fn describe_hir_body_shape(db: &dyn HirDb, body: Body<'db>, sink: &mut impl ShapeSink) ->
Result<()> {
    // HIR owns which HIR nodes exist and how they should be represented.
    // The shape engine only receives keys, kinds, fields, children, and edges.
}

```

The derive macro should return later, but phase adapters should come first. That avoids locking every IR into a proc-macro policy before the dimensions and cycle rules are proven.

Shape facts for trace integration

```

pub enum TraceFact {
    // existing facts ...
    ShapePolicy(ShapePolicyFact),
    ShapeNodeHash(ShapeNodeHashFact),
    ShapeComponentHash(ShapeComponentHashFact),
    ShapeGraphHash(ShapeGraphHashFact),
}

```

```

}

pub struct ShapeNodeHashFact {
    pub node: OriginExportKey,
    pub graph: ShapeGraphKey,
    pub policy: ShapePolicyId,
    pub local: DimensionDigests,
    pub tree: DimensionDigests,
    pub component: Option<DimensionDigests>,
}

pub struct ShapeGraphHashFact {
    pub graph: ShapeGraphKey,
    pub owner: OriginExportKey,
    pub policy: ShapePolicyId,
    pub dimensions: DimensionDigests,
}

```

Shape facts are derived facts. Trace validation should ensure their node keys reference known origin nodes when they describe real compiler entities.

7. Implementation plan

This should be a clean phased push after the current cleanup branch stabilizes. Do not try to add Datalog, DWARF, ethdebug, LSP, and fuzzing in the same PR as the core hash engine.

Phase	Scope	Deliverables	Acceptance gate
0. Spec and golden fixtures	Write the hash schema spec before coding.	Policy version, dimension definitions, edge taxonomy, codec spec, digest algorithm choice, fixture corpus.	A reviewer can predict which dimension changes for each fixture mutation.
1. Core shape graph types	Validated generic data model.	ShapeGraph, ShapeNodeKey, ShapeField, ShapeChild, ShapeEdge, ShapeHashPolicy, digest type.	No phase semantics in the crate; invalid labels/keys rejected.
2. Local + acyclic Merkle hashes	Rebuild old local/tree functionality with a stronger digest.	local_hash, ordered_child_hash, node_tree_hash, graph root for acyclic child graph.	Old tests ported and passing; child cycles rejected clearly.
3. Graph-edge aggregation	Restore graph-wide aggregation over sorted nodes/edges.	graph_digest by dimension; label-sensitive edge contribution; identity-bound and anonymous modes separated.	Full edge-label and insertion-order tests pass.
4. Cycle-aware component hashing	Support cyclic IR graphs deliberately.	Tarjan SCC condensation; component hashes;	Cyclic CFG fixture hashes deterministically and

		graph digest over condensation DAG; optional WL mode behind feature flag.	does not recurse forever.
5. Phase adapters	Describe real IRs.	HIR first, then MIR, then Sonatina/codegen; no derive macro yet.	Shape facts emitted from real compiler data for a small fixture.
6. Trace-facts integration	Expose hashes to tooling.	ShapePolicyFact, ShapeNodeHashFact, ShapeGraphHashFact, validation.	fe trace validate accepts shape facts and rejects invalid endpoints/policies.
7. CLI demos	Make it useful quickly.	fe shape emit, fe shape diff, fe shape explain-node, fe shape bucket.	Can show rename-only, literal-only, and CFG-only changes on fixtures.
8. Derive macro reintroduction	Ergonomics after policy stabilizes.	ShapeDescribe derive requiring explicit field policy and skip reasons.	Compile-fail tests catch unclassified fields and empty skip reasons.
9. Fuzzing integration	Use the hashes as corpus/introspection tools.	Shape-bucketed corpus dedup, mutation targeting, coverage by dimensions.	Fuzzer can avoid redundant shape-equivalent cases under chosen policy.

Minimal demos

- **Demo A - AST/HIR tree Merkle hash:** two functions differing only in literal values should have equal Structure and Names digests but different Constants digest.
- **Demo B - rename elision:** two functions differing only in local names should have equal Structure and Constants digests but different Names digest.
- **Demo C - CFG cyclic hash:** a MIR loop with a backedge should hash deterministically under CondenseScc and reject under Reject if modeled as recursive dependency edges.
- **Demo D - insertion-order stability:** build the same graph in two different insertion orders and get identical graph hashes.
- **Demo E - edge-label sensitivity:** changing cfg:then to cfg:else changes Structure graph digest even if all nodes are identical.
- **Demo F - fuzz bucketing:** generate variants of a small expression and show buckets by structure-only, constants-only, and exact digests.

8. Tests and acceptance criteria

A content-addressing system is only as good as its regression tests. Port the old tests, then add tests for the architectural fixes.

Test family	Required cases
Validation	Reject empty keys/kinds/field names/edge labels; reject duplicate node keys; reject missing endpoints; reject invalid digest/policy IDs.

Dimension purity	Constants changes do not alter Structure; names changes do not alter Constants; child label changes alter Structure only; type changes alter Types only unless policy says otherwise.
Ordering	Unordered fields are insertion-order independent; ordered children are order-sensitive; graph edges are insertion-order independent.
Graph edges	Graph edges do not suppress child content hashing; graph digest uses full edge labels; edge endpoints contribute according to policy.
Identity vs anonymous	Changing stable node keys changes identity-bound digest but not anonymous structure digest when shape is otherwise the same.
Cycles	Reject mode errors on child cycles; SCC mode hashes cyclic graph deterministically; cyclic edge labels and direction affect digest; insertion order does not.
Hash algorithm/versioning	Changing schema version, algorithm, dimension policy, or view mode changes policy ID and durable hashes.
Phase integration	Same local ID under two owners produces distinct identity-bound hashes; phase adapters do not use raw IDs or source spans as identity.
Fuzzing	Generated cases bucket as expected under structure-only and exact policies; collisions are reported as hash equality, not semantic equality.

Hard acceptance checklist

- Every real shape node references a canonical OriginExportKey or a derived key owned by an OriginExportKey.
- No field is included in a hash dimension without an explicit policy.
- Hash algorithm and schema version are visible in every exported hash policy.
- Acyclic and cyclic graph behavior is explicit and tested.
- The shape layer can be deleted without deleting compiler truth; only derived views disappear.
- The old graph-edge bugs stay fixed: edges do not suppress child content and labels are not truncated.

9. Risks, caveats, trade-offs, and workarounds

Risk / caveat	Why it matters	Recommended workaround
SSOT regression	A content-address graph can become a hidden identity system.	Use OriginExportKey for entity keys; mark shape facts as derived; phase crates own adapters.
Cycles are not Merkle DAGs	Recursive content addressing cannot directly solve cyclic	Use acyclic child edges plus graph-edge aggregation, or SCC

	dependencies.	condensation.
WL hashes are approximate	WL-style graph hashes are useful but not exact graph canonicalization.	Label them as anonymous structural fingerprints; use exact canonical labeling only if needed later.
64-bit digest collisions	Current/old FNV-size hashes are too small for durable content addressing.	Use BLAKE3-256 internally or SHA-256/multihash externally; keep short display hashes only as UI aliases.
Dimension bleed	Constants/names/types accidentally changing structure makes elision useless.	Policy tests for every dimension; derive macro requires field annotations; skip reasons required.
Hash churn	Minor implementation details can invalidate caches and reports.	Version policies; use golden fixtures; document intentional churn.
Performance	Large IR graphs may make full graph hashing expensive.	Cache local hashes; incremental recompute; hash per function/body; lazy CLI reports.
Overclaiming	Shape equality can be mistaken for semantic equivalence.	Reports must say “same under shape policy”, not “semantically same”.
JSON instability	Non-canonical JSON can break signing/re-hashing.	Use binary hash codec; for JSON artifacts, use canonical JSON or include precomputed digests only.
Proc macro opacity	A derive macro can hide policy errors.	Start with manual adapters; re-add derive only with compile-fail tests and generated policy audit.

10. Fuzzing, trace, CLI/LSP, Datalog, and proof-facing integrations

This is why the shape-addressing system is worth bringing back. It can become the backbone of granular compiler probing without undermining the SSOT invariant.

Granular fuzzing

- Use Structure hashes to deduplicate alpha-renamed or literal-mutated variants that have the same topology.
- Use Constants hashes to deliberately target literal-dependent behavior without changing control shape.
- Use Types hashes to bucket type-system or ABI-layout cases.
- Use TraceEvents hashes to detect instrumentation/compiler-event drift between optimization passes.
- Use per-node hashes to minimize failing cases: preserve the failing aggregate hash while shrinking unrelated subgraphs.

```

fe fuzz run --bucket-by shape:hir:structure
fe fuzz shrink crash.fe --preserve-shape mir:structure --preserve-origin culprit-node
fe shape diff before.trace.jsonl after.trace.jsonl --dimensions structure,names,constants,types

```

Trace and Datalog

Shape hashes should be emitted as trace facts after validation. Datalog should consume them as base facts and derive queries such as “which source ranges changed only by constants?” or “which MIR loops have the same structure as this expensive loop?” The relation schema should be generated from typed ShapeHash facts; no manual relation mirror.

```

base_shape_node_hash(Node, Graph, Policy, Dimension, Local, Tree, Component).
base_shape_graph_hash(Graph, Policy, Dimension, Digest).

```

```

same_structure_graph(A, B) :-
    base_shape_graph_hash(A, Policy, "structure", D),
    base_shape_graph_hash(B, Policy, "structure", D).

```

```

literal_only_change(A, B) :-
    same_structure_graph(A, B),
    same_names_graph(A, B),
    base_shape_graph_hash(A, Policy, "constants", DA),
    base_shape_graph_hash(B, Policy, "constants", DB),
    DA != DB.

```

CLI and LSP

- **CLI first:** fe shape emit/diff/explain/bucket should prove usefulness before LSP UI work.
- **LSP hovers:** show “structure hash changed; constants same; names changed” for a selected function or loop.
- **Inlays:** keep to short display fingerprints and warnings, not full hash material.
- **HTTP/browser graph:** use OriginExportKey and shape hashes to navigate source, IR, trace events, and content-address nodes.

Proof-facing and attestation uses

Shape/content addresses can support audit trails and proof inputs, but they are not themselves proofs of semantic preservation. Good proof-facing claims are narrow: “this artifact contains a node/subgraph with this declared structure hash”, “this compiled artifact was generated under policy P from trace T”, or “this required bottleneck/control-flow shape exists under the declared graph policy”. Avoid broad claims such as “semantics preserved” unless a separate formal proof establishes the exact property.

Appendix A. Old implementation review notes

Files reviewed on the origin-overhaul branch:

- crates/common/src/shape.rs: <https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/common/src/shape.rs>
- crates/common/src/shape/graph.rs: <https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/common/src/shape/graph.rs>
- crates/common/src/shape/hash.rs: <https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/common/src/shape/hash.rs>
- crates/common/src/shape/describe.rs: <https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/common/src/shape/describe.rs>
- crates/common/src/shape/field_value.rs: https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/common/src/shape/field_value.rs

- crates/shape-derive/src/lib.rs:
<https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul/crates/shape-derive/src/lib.rs>
- crates/common/src/shape.rs on origin-overhaul-phased:
<https://raw.githubusercontent.com/micahscopes/fe/origin-overhaul-phased/crates/common/src/shape.rs>

Old ShapeGraph

- ShapeDimension had five projections: Structure, Names, Constants, Types, TraceEvents.
- ShapeNodeId was a compact u32 wrapper indexing nodes in a ShapeGraph.
- ShapeNode held a stable_key, kind, fields, and children.
- ShapeField held a dimension, name, and value.
- ShapeChild held a label and child node; child order was preserved by Vec order.
- ShapeEdge held from/to node IDs and a label; graph edges were separate from child containment.
- ShapeGraph enforced non-empty keys/kinds/labels and duplicate stable-key rejection.

Old hash engine

- StableDigest was a 64-bit FNV-1a style digest.
- DimensionDigests was a fixed array indexed by ShapeDimension.
- local_digests included node kind only in Structure and sorted fields within their own dimensions.
- tree_digests recursively hashed children; labels contributed to Structure; child order was preserved; cycles caused a panic.
- graph_digests sorted nodes by stable_key and edges by source key, label, target key. Nodes contributed their tree digest for every dimension. Graph edges contributed only in Structure and used Structure endpoint hashes.
- Tests covered the important old bugs: graph edges must not suppress child content, graph labels must not be truncated, and fields are unordered while children are ordered.

Old derive macro

- ShapeDescribe derive accepted container/variant kind and stable_key attributes.
- Every field had to declare a shape policy: field dimension, child, custom with function, or skip with a non-empty reason.
- The macro rejected missing field policy, duplicate attrs, empty kind/label, duplicate labels, and skip with empty reason.
- This strictness is worth reintroducing later, after the core hash model and cycle policy stabilize.

Appendix B. Research notes and references

Research notes used to sound the design:

Reference	Design relevance	URL
IPFS Merkle DAG docs	Merkle DAG node identifiers are hashes of node payload plus child identifiers; construction is leaf-first; nodes are immutable; cycles cannot be constructed directly in ordinary Merkle DAGs.	https://docs.ipfs.tech/concepts/merkle-dag/
NetworkX Weisfeiler-Lehman	WL hashing iteratively	https://networkx.org/

graph hash docs	aggregates node neighborhoods; it supports edge/node labels and subgraph hashes, but hash/version behavior is policy-sensitive and should not be treated as exact semantic equivalence.	documentation/stable/reference/algorithms/generated/networkx.algorithms.graph_hashing.weisfeiler_lehman_graph_hash.html
NetworkX WL subgraph hash docs	WL subgraph hashes provide per-node depth-neighborhood fingerprints, which is useful for fuzzing/search but depends on iteration depth and labels.	https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.graph_hashing.weisfeiler_lehman_subgraph_hashes.html
RFC 8785 JSON Canonicalization Scheme	Cryptographic hashing/signing requires invariant representation; JCS defines deterministic JSON property sorting and strict data constraints. Use this only for JSON exports or use a canonical binary codec for internal hashes.	https://www.rfc-editor.org/rfc/rfc8785
BLAKE3 Rust crate docs	BLAKE3 provides 32-byte output, keyed hashing, and derive-key/context-style APIs. If used, domain separation and schema versioning should be explicit.	https://docs.rs/blake3/latest/blake3/

Appendix C. Agent checklist

Directive to the implementor agent:

Do not expand unrelated tooling while implementing content addressing.

First PR:

- Add generic shape-address data model.
- Implement local + acyclic tree hashing with strong digest and domain separation.
- Port old tests.
- Add identity-bound vs anonymous mode tests.
- Reject cycles clearly.

Second PR:

- Add graph-edge aggregation.
- Add insertion-order stability tests.
- Add full-label tests.

Third PR:

- Add SCC cycle policy.
- Add cyclic CFG fixture.
- Add shape facts to trace-facts.

Fourth PR:

- Add HIR and MIR adapters.
- Add CLI demos: emit, diff, bucket.

Only after that:

- Reintroduce ShapeDescribe derive.
- Add Datalog shape rules.
- Add LSP/browser UX.
- Integrate with fuzzing corpus management.

Reviewer stop sign

Reject any implementation that uses raw local IDs or source spans as shape identity, writes semantic phase variants into common, silently hashes cyclic recursive edges, or treats shape equality as semantic equivalence.